

UNIVERSIDAD PERUANA
CAYETANO HEREDIA



FACULTAD DE CIENCIAS E INGENIERÍA
“ALBERTO CAZORLA TALLERÍ”
Departamento Académico de Ciencias Exactas - Sección Ingeniería

INFORME 9

CURSO
SISTEMAS OPERATIVOS

INTEGRANTES

-
- Huancas Bereche, Erick Emilio
- Pellanne Aro, Carlos Alfredo
- Reyes Garcia Alvaro Pablo
- Rivera Garayar, Matias Gabriel
- Romero Chac, Luis Antonio

DOCENTE

Dr. Roberto Eduardo Burga Montañez

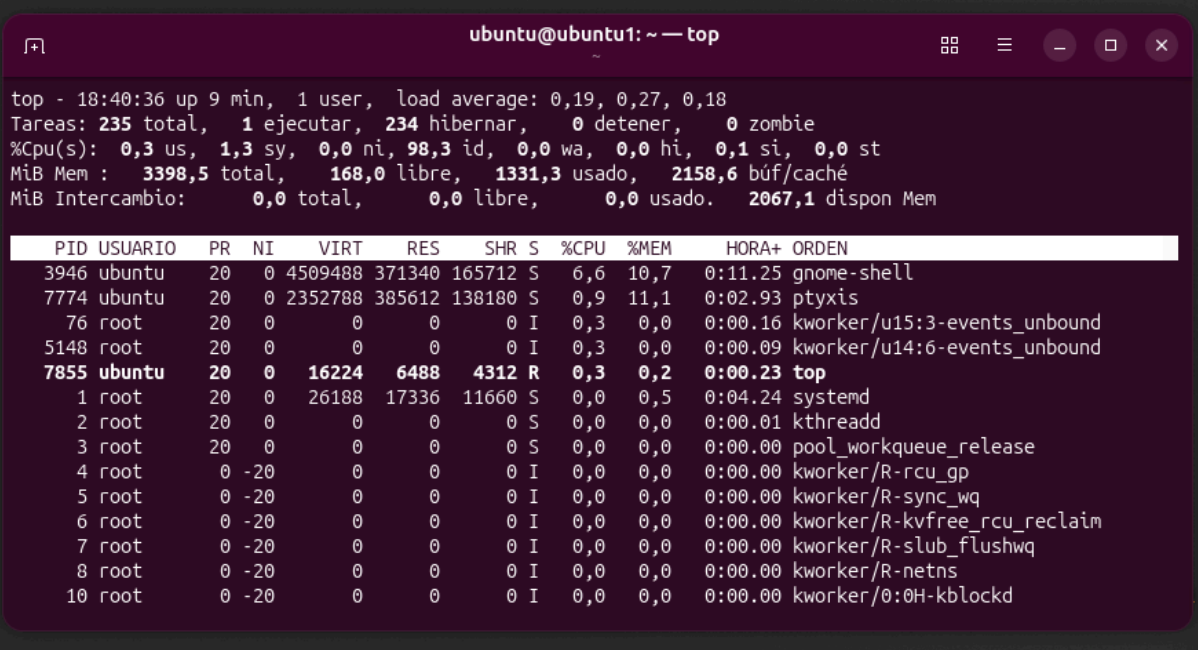
GRUPO
SO2 - G2

ACTIVIDAD 1: Observando procesos en Linux

Objetivo: Identificar procesos reales y su comportamiento.

Comandos:

top: El comando top en Linux sirve para ver en tiempo real qué procesos están consumiendo recursos del sistema.



```
top - 18:40:36 up 9 min, 1 user, load average: 0,19, 0,27, 0,18
Tareas: 235 total, 1 ejecutar, 234 hibernar, 0 detener, 0 zombie
%Cpu(s): 0,3 us, 1,3 sy, 0,0 ni, 98,3 id, 0,0 wa, 0,0 hi, 0,1 si, 0,0 st
MiB Mem : 3398,5 total, 168,0 libre, 1331,3 usado, 2158,6 búf/caché
MiB Intercambio: 0,0 total, 0,0 libre, 0,0 usado. 2067,1 dispon Mem
```

PID	USUARIO	PR	NI	VIRT	RES	SHR	S	%CPU	%MEM	HORA+	ORDEN
3946	ubuntu	20	0	4509488	371340	165712	S	6,6	10,7	0:11.25	gnome-shell
7774	ubuntu	20	0	2352788	385612	138180	S	0,9	11,1	0:02.93	ptxis
76	root	20	0	0	0	0	I	0,3	0,0	0:00.16	kworker/u15:3-events_unbound
5148	root	20	0	0	0	0	I	0,3	0,0	0:00.09	kworker/u14:6-events_unbound
7855	ubuntu	20	0	16224	6488	4312	R	0,3	0,2	0:00.23	top
1	root	20	0	26188	17336	11660	S	0,0	0,5	0:04.24	systemd
2	root	20	0	0	0	0	S	0,0	0,0	0:00.01	kthreadd
3	root	20	0	0	0	0	S	0,0	0,0	0:00.00	pool_workqueue_release
4	root	0	-20	0	0	0	I	0,0	0,0	0:00.00	kworker/R-rcu_gp
5	root	0	-20	0	0	0	I	0,0	0,0	0:00.00	kworker/R-sync_wq
6	root	0	-20	0	0	0	I	0,0	0,0	0:00.00	kworker/R-kvfree_rcu_reclaim
7	root	0	-20	0	0	0	I	0,0	0,0	0:00.00	kworker/R-slub_flushwq
8	root	0	-20	0	0	0	I	0,0	0,0	0:00.00	kworker/R-netns
10	root	0	-20	0	0	0	I	0,0	0,0	0:00.00	kworker/0:0H-kblockd

En este caso podemos observar que procesador se encuentra prácticamente libre, con aproximadamente un 98,3 % de CPU inactiva, mientras que el proceso que más recursos utiliza es gnome-shell, con un 6,6 % de CPU. En cuanto a la memoria, el sistema dispone de aproximadamente 3,4 GB de RAM, de los cuales una parte se encuentra utilizada y otra se utiliza como caché. El load average es bajo, con valores de 0,19, 0,27 y 0,18, lo que indica que no existen procesos generando una carga significativa.

htop

```
ubuntu@ubuntu1: ~ — htop

0.7% Tasks: 123, 411 thr, 100 kthr; 1 running
1.4% Load average: 0.09 0.09 0.11
0.0% Uptime: 00:20:11
Mem[|||||||||||||||||||||||||||||||||1.13G/3.32G]
Swp[|||||0K/0K]

Main I/O
PID USER PRI NI VIRT RES SHR S CPU% MEM% TIME+ Command
4086 ubuntu 20 0 4402M 364M 0 S 0.7 10.7 0:02.00 /usr/bin/gnome-shell --mode=ubuntu
4087 ubuntu 20 0 4402M 364M 0 S 0.7 10.7 0:02.33 /usr/bin/gnome-shell --mode=ubuntu
8286 ubuntu 20 0 14216 5352 3976 R 0.7 0.2 0:01.28 htop
1 root 20 0 26188 17336 11660 S 0.0 0.5 0:04.28 /usr/lib/systemd/systemd --switched-roo
357 root 19 -1 67368 18404 16544 S 0.0 0.5 0:00.34 /usr/lib/systemd/systemd-journald
402 systemd-oo 20 0 16808 7760 6688 S 0.0 0.2 0:00.33 /usr/lib/systemd/systemd-oamd
403 systemd-re 20 0 22548 14996 12380 S 0.0 0.4 0:00.15 /usr/lib/systemd/systemd-resolved
405 root 20 0 41356 14992 9268 S 0.0 0.4 0:00.27 /usr/lib/systemd/systemd-udev
1086 avahi 20 0 6744 4832 4448 S 0.0 0.1 0:00.07 avahi-daemon: running [ubuntu1.local]
1088 root 20 0 2888 2004 1884 S 0.0 0.1 0:00.01 /bin/sh /usr/lib/systemd/scripts/chrony
1089 messagebus 20 0 11904 8900 4788 S 0.0 0.3 0:01.26 @dbus-daemon --system --address=systemd

F1Help F2Setup F3Search F4Filter F5Tree F6SortBy F7Nice F8Nice+ F9Kill F10Quit
```

La función de htop es mostrar de manera interactiva y en tiempo real el estado de los recursos del sistema y los procesos que se están ejecutando.

ps -aux

```
ubuntu@ubuntu1:~$ ps -aux
USER          PID %CPU %MEM    VSZ   RSS TTY      STAT START   TIME COMMAND
root            1  0.2  0.4 26188 17336 ?        Ss   18:30   0:04 /usr/lib/systemd/systemd --switched-roo
root            2  0.0  0.0      0      0 ?        S    18:30   0:00 [kthreadd]
root            3  0.0  0.0      0      0 ?        S    18:30   0:00 [pool_workqueue_release]
root            4  0.0  0.0      0      0 ?        I<   18:30   0:00 [kworker/R-rcu_gp]
root            5  0.0  0.0      0      0 ?        I<   18:30   0:00 [kworker/R-sync_wq]
root            6  0.0  0.0      0      0 ?        I<   18:30   0:00 [kworker/R-kvfree_rcu_reclaim]
root            7  0.0  0.0      0      0 ?        I<   18:30   0:00 [kworker/R-slub_flushwq]
root            8  0.0  0.0      0      0 ?        I<   18:30   0:00 [kworker/R-netns]
root           10  0.0  0.0      0      0 ?        I<   18:30   0:00 [kworker/0:0H-kblockd]
root           11  0.0  0.0      0      0 ?        I    18:30   0:00 [kworker/0:1-mm_percpu_wq]
root           12  0.0  0.0      0      0 ?        I    18:30   0:00 [kworker/u12:0-ipv6_addrconf]
root           13  0.0  0.0      0      0 ?        I<   18:30   0:00 [kworker/R-mm_percpu_wq]
root           14  0.0  0.0      0      0 ?        S    18:30   0:00 [ksoftirqd/0]
root           15  0.0  0.0      0      0 ?        I    18:30   0:00 [rcu_preempt]
root           16  0.0  0.0      0      0 ?        S    18:30   0:00 [rcu_exp_par_gp_kthread_worker/0]
root           17  0.0  0.0      0      0 ?        S    18:30   0:00 [rcu_exp_gp_kthread_worker]
root           18  0.0  0.0      0      0 ?        S    18:30   0:00 [migration/0]
root           19  0.0  0.0      0      0 ?        S    18:30   0:00 [kprobe-optimizer]
root           20  0.0  0.0      0      0 ?        S    18:30   0:00 [idle_inject/0]
root           21  0.0  0.0      0      0 ?        S    18:30   0:00 [causal/0]
```

ACTIVIDAD 2: Simulando prioridades(nice)

Objetivo: Entender planificación por prioridades.

Ejecución:

- Abrir terminal y correr:
- `nice -n 10 yes > /dev/null &`

```
ubuntu@ubuntu1:~$ nice -n 10 yes > /dev/null &  
[1] 8547
```

- `nice -n -5 yes > /dev/null &`

```
ubuntu@ubuntu1:~$ nice -n -5 yes > /dev/null &  
[1] 4820
```

ACTIVIDAD 3: Round Robin en Linux(conceptual + práctica)

Objetivo: Relacionar Linux con Round Robin.

Ejercicio Ejecutar varios procesos:

- `yes > /dev/null &`

```
ubuntu@ubuntu1:~/Escritorio$ yes > /dev/null &  
[1] 6459
```

- `yes > /dev/null &`

```
ubuntu@ubuntu1:~/Escritorio$ yes > /dev/null &  
[2] 6507
```

- `yes > /dev/null &`

```
ubuntu@ubuntu1:~/Escritorio$ yes > /dev/null &  
[3] 6509
```

Luego observar con:

- `top`

```
top - 18:50:29 up 14 min, 1 user, load average: 2,54, 2,38, 1,46  
Tareas: 222 total, 1 ejecutar, 220 hibernar, 0 detener, 1 zombie  
%Cpu(s): 0,3 us, 5,0 sy, 0,0 ni, 94,2 id, 0,0 wa, 0,0 hi, 0,5 si, 0,0 st  
MiB Mem : 3398,5 total, 373,0 libre, 1273,1 usado, 2007,0 búf/caché  
MiB Intercambio: 0,0 total, 0,0 libre, 0,0 usado. 2125,3 dispon Mem
```

PID	USUARIO	PR	NI	VIRT	RES	SHR	S	%CPU	%MEM	HORA+	ORDEN
3800	ubuntu	20	0	4502908	352552	162992	S	8,3	10,1	0:45.44	gnome-shell
6386	ubuntu	20	0	2332844	360304	140124	S	2,3	10,4	0:09.04	ptxixis
875	root	20	0	18588	9756	8300	S	0,3	0,3	0:01.40	systemd-logind
4058	ubuntu	20	0	390916	11932	8012	S	0,3	0,3	0:02.95	ibus-daemon
4094	ubuntu	20	0	649244	35560	27872	S	0,3	1,0	0:00.32	update-notifier
6512	root	20	0	0	0	0	I	0,3	0,0	0:00.68	kworker/2:1-events
1	root	20	0	25932	17316	11684	S	0,0	0,5	0:08.32	systemd
2	root	20	0	0	0	0	S	0,0	0,0	0:00.06	kthreadd
3	root	20	0	0	0	0	S	0,0	0,0	0:00.00	pool_workqueue_release
4	root	0	-20	0	0	0	I	0,0	0,0	0:00.00	kworker/R-rcu_gp
5	root	0	-20	0	0	0	I	0,0	0,0	0:00.00	kworker/R-sync_wq
6	root	0	-20	0	0	0	I	0,0	0,0	0:00.00	kworker/R-kvfree_rcu_reclaim
7	root	0	-20	0	0	0	I	0,0	0,0	0:00.00	kworker/R-slub_flushwq
8	root	0	-20	0	0	0	I	0,0	0,0	0:00.00	kworker/R-netns
10	root	0	-20	0	0	0	I	0,0	0,0	0:00.00	kworker/0:0H-kblockd

ACTIVIDAD 4: Cambiando prioridades en vivo

Objetivo: Ver cómo afecta la planificación.

Ejecución:

1. Ejecutar proceso: `yes > /dev/null &`

```
ubuntu@ubuntu1:~/Escritorio$ yes > /dev/null &  
[1] 6567
```

El comando `yes > /dev/null &` permite ejecutar el proceso `yes` en segundo plano. El programa `yes` genera continuamente el carácter `y`, pero mediante `> /dev/null` toda esta salida se redirige al dispositivo `/dev/null`, evitando que se muestre en la terminal. El símbolo `&` indica que el proceso debe ejecutarse en segundo plano, por lo que la terminal queda disponible para introducir otros comandos. Al ejecutarlo, el sistema asigna un PID (Process ID) al proceso, que posteriormente será necesario para identificarlo y modificar su prioridad.

2. Obtener PID: `ps -aux | grep yes`

```
ubuntu@ubuntu1:~/Escritorio$ ps -aux | grep yes  
ubuntu      6567 99.2  0.2  16244  7312 pts/0    R   18:52   0:38 yes  
ubuntu      6573  0.0  0.0   11932  2692 pts/0    S+  18:53   0:00 grep --color=auto yes
```

El comando `ps -aux | grep '[y]es'` se utiliza para localizar el proceso `yes` que se encuentra ejecutándose. La instrucción `ps -aux` muestra información sobre los procesos activos del sistema, mientras que el operador `|` permite enviar esa información al comando `grep`. Finalmente, `grep '[y]es'` filtra los resultados para mostrar únicamente la línea correspondiente al proceso `yes`. Entre los datos mostrados se encuentra el PID, que es el identificador único del proceso. Por ejemplo, si aparece el número 12345, ese será el PID que se utilizará en el siguiente paso.

3. Cambiar prioridad: `renice -n 15 -p PID`

```
ubuntu@ubuntu1:~/Escritorio$ renice -n 15 -p 6567  
6567 (process ID) prioridad anterior 0, nueva prioridad 15
```

El comando `renice -n 15 -p 12345` permite modificar la prioridad del proceso identificado mediante su PID. La instrucción `renice` cambia el valor `nice` de un proceso que ya está en ejecución; `-n 15` establece el nuevo valor en 15, mientras que `-p 12345` indica que el cambio se aplicará al proceso cuyo PID es 12345. En este caso, aumentar el valor `nice` de 0 a 15 hace que el proceso tenga una prioridad de CPU relativamente menor y, por tanto, ceda más tiempo de procesador a procesos con valores `nice` inferiores. El resultado del comando normalmente informa de la prioridad anterior y de la nueva prioridad asignada.

ACTIVIDAD 5: Identificando tipos de procesos

Objetivo: Diferenciar CPU-bound vs I/O-bound

Ejercicio Ejecutar:

- `yes > /dev/null &`

```
ubuntu@ubuntu1:~$ yes > /dev/null &  
[1] 5124  
ubuntu@ubuntu1:~$
```

- `sleep 100 &`

```
ubuntu@ubuntu1:~$ sleep 100 &  
[2] 6463  
ubuntu@ubuntu1:~$
```

ACTIVIDAD 6: Ejercicio

Ejercicio: Ejecuta:

- `yes > /dev/null &`

```
ubuntu@ubuntu1:~$ yes > /dev/null &  
[3] 9330  
[2] Hecho sleep 100  
ubuntu@ubuntu1:~$
```

- `yes > /dev/null &`

```
ubuntu@ubuntu1:~$ yes > /dev/null &  
[4] 17985  
ubuntu@ubuntu1:~$
```

- `nice -n 10 yes > /dev/null &`

```
ubuntu@ubuntu1:~$ nice -n 10 yes > /dev/null &  
[5] 18483  
ubuntu@ubuntu1:~$
```

REFERENCIAS BIBLIOGRÁFICAS:

- 1-.Linux man-pages project. (2026). kill(2) — Linux manual page. Linux man-pages.
<https://man7.org/linux/man-pages/man2/kill.2.html>
- 2-. Debian Project. (2026). apt(8) — command-line interface. Debian Manpages.
<https://manpages.debian.org/trixie/apt/apt.8.en.html>
- 3-. Ubuntu Documentation. (2022). Installing software. Ubuntu.
<https://help.ubuntu.com/community/InstallingSoftware>

